TP 2 — Premiers conteneurs avec Docker

Binôme : DIARRA ABDOULAYE · BAMBA IDRISS · SORO SANGUINE VICTOIRE LOUISE
Date : 29/10/2026
Cours : Introduction au Cloud Computing, L3 RIST / BDGL
Image publiée : `docker.io/bambaidrissshafihi/api-fil-rouge:1.1` (dépôt **public**)
Digest : `sha256:f85fb2d13ba54447edd49976aab656ae1bf3bc68c286fbaa145e537e2de95521`

> Les captures des sorties de commandes sont intégrées à la fin de chaque partie (figures numérotées).

Partie 0 — Préparer la VM et installer Docker

Redirections VirtualBox créées (`web` 8080→8080, `api` 5000→5000, `ssh` 2222→22), Docker installé avec `sudo apt install -y docker.io docker-buildx docker-compose-v2`, puis `sudo usermod -aG docker etudiant`. `docker version`, `docker info`.

Élément	Valeur relevée
Docker (client et serveur)	29.1.3 (API 1.52, Go 1.24.13)
containerd / runc	2.2.2 / 1.4.0
Storage Driver	overlayfs
Cgroup	systemd, version 2
Kernel Version	7.0.0-38-generic (Ubuntu 26.04.1 LTS)

Q1. Docker 29.1.3. Le noyau utilisé est `7.0.0-38-generic`, celui de la VM : Docker n'a pas de noyau propre, il s'appuie sur celui de l'hôte.

Q2. Le démon Docker tourne en root. Un membre du groupe `docker` peut lancer un conteneur qui monte `/` de la machine (`docker run -v /:/hote …`) et lire ou modifier n'importe quel fichier : il devient root de fait. Sur un serveur partagé, on n'ajoute au groupe que des personnes de confiance.

Captures des sorties

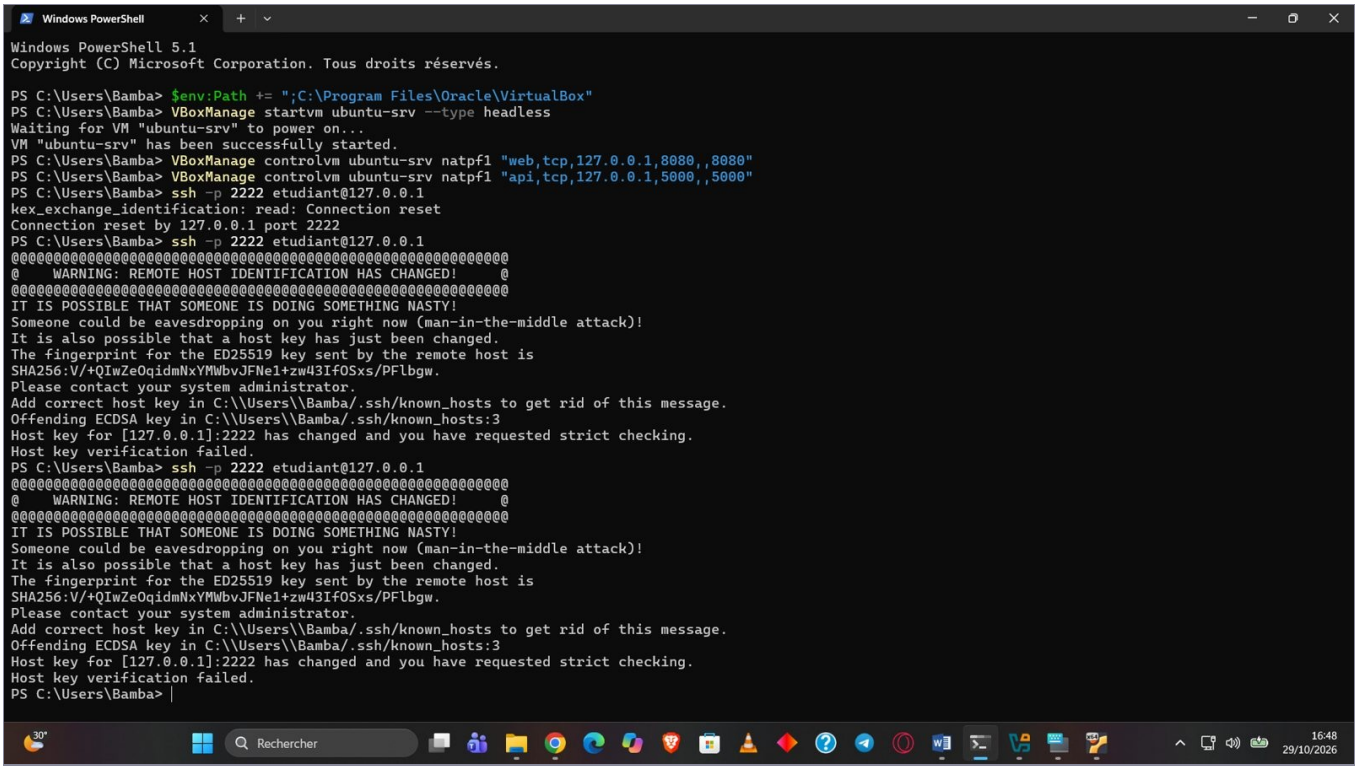


Figure 1. PowerShell : démarrage de la VM et création des redirections de ports

```
Command 'tdre' not found, did you mean:
  command 'tdr' from deb devtodo (0.1.20+git20200830.0ad52b0-3build1)
  command 'tre' from deb tre-command (0.4.0-7)
  command 'tde' from deb devtodo (0.1.20+git20200830.0ad52b0-3build1)
Try: sudo apt install <deb name>
etudiant@ubuntu-srv:~$ sudo usermod -aG etudiant
Utilisation : usermod[options] LOGIN

Options :
  -a, --append                  ajouter l'utilisateur au GROUPS supplémentaire
                                      mentionné par l'option -G sans supprimer
                                      l'utilisateur des autres groupes
  -b, --badname                 allow bad names (DEPRECATED)
  -c, --comment COMMENT        nouvelle valeur du champ GECOS
  -d, --home HOME_DIR           nouveau répertoire personnel du compte utilisateur
 -e, --expiredate EXPIRE_DATE  définir la date d'expiration du compte avec EXPIRE_DATE
  -f, --inactive INACTIVE       définir le mot de passe inactif après expiration
                                à INACTIVE
  -g, --gid GROUP               forcer l'utiisation de GROUP comme groupe primaire
  -G, --groups GROUPS           nouvelle liste de groupes GROUPS supplémentaires
  -h, --help                   afficher ce message d'aide et quitter
  -l, --login NEW_LOGIN         nouvelle valeur pour le nom de connexion
  -L, --lock                    verrouiller le compte utilisateur
  -m, --move-home               déplacer le contenu du répertoire personnel dans le
                                    nouvel emplacement (à utiliser seulement avec -d)
  -o, --non-unique              autoriser l'utilisation d'UID dupliqué (non-unique)
  -p, --password PASSWORD       utiliser un mot de passe chiffré pour le nouveau mot de passe
  -P, --prefix PREFIX_DIR       préfixe du répertoire où sont situés les fichiers etc/*
  -r, --remove                   supprimer l'utilisateur uniquement du GROUPS supplémentaire
                                      mentionné par l'option -G sans supprimer
                                      l'utilisateur des autres groupes
 -R, --root CHROOT_DIR          répertoire dans lequel faire un chroot
  -s, --shell SHELL             nouveau shell de login pour le compte utilisateur
  -u, --uid UID                nouvel UID pour le compte utilisateur
  -U, --unlock                  déverrouiller le compte utilisateur
  -v, --add-subuids FIRST-LAST  ajouter une gamme d'UID subordonnés
  -V, --del-subuids FIRST-LAST  supprimer la gamme des UID subordonnés
  -w, --add-subgids FIRST-LAST  ajouter une gamme de GID subordonnés
  -W, --del-subgids FIRST-LAST supprimer la gamme des GID subordonnés
 -Z, --selinux-user SEUSER   nouveau mappage d'utilisateur SELinux pour le compte utilisateur
     --selinux-range SERANGE   nouvelle plage MLS de SELinux pour le compte utilisateur

etudiant@ubuntu-srv:~$ getent group docker
docker:x:107:
etudiant@ubuntu-srv:~$ [ 2809.108981] watchdog: BUG: soft lockup - CPU#0 stuck for 200s! [swapper/0:0]
getent group dockerwxqs
etudiant@ubuntu-srv:~$ sudo usermod -aG docker etudiant
etudiant@ubuntu-srv:~$ getent group docker
docker:x:107:etudiant
etudiant@ubuntu-srv:~$ _
```

Figure 2. VM : ajout de l'utilisateur au groupe docker (`getent group docker`)

```
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe55:89e8


La maintenance de sécurité étendue pour Applications n'est pas activée.

31 mises à jour peuvent être appliquées immédiatement.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

Activez ESM Apps pour recevoir des futures mises à jour de sécurité supplémentaires.
Visitez https://ubuntu.com/esm ou executez : sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings


etudiant@ubuntu-srv:~$ docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.24.13
 Git commit:        29.1.3-0ubuntu4.1
 Built:             Wed Apr 29 16:40:20 2026
 OS/Arch:           linux/amd64
 Context:           default

Server:
 Engine:
  Version:          29.1.3
  API version:      1.52 (minimum version 1.44)
  Go version:       go1.24.13
  Git commit:       29.1.3-0ubuntu4.1
  Built:            Wed Apr 29 16:40:20 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          2.2.2
  GitCommit:
 runc:
  Version:          1.4.0-0ubuntu1
  GitCommit:
 docker-init:
  Version:          0.19.0
  GitCommit:
etudiant@ubuntu-srv:~$ docker info | grep -E "Server Version|Storage Driver|Cgroup|Kernel Version"
 Server Version: 29.1.3
 Storage Driver: overlayfs
 Cgroup Driver: systemd
 Cgroup Version: 2
 Kernel Version: 7.0.0-38-generic
etudiant@ubuntu-srv:~$
```

Figure 3. `docker version` (client et serveur) et `docker info` : overlayfs, cgroup v2, noyau 7.0.0-38-generic

Partie 1 — Lancer et inspecter des conteneurs

Commandes : `docker run hello-world`, `docker run -d --name web -p 8080:80 nginx`, `docker exec`, `docker logs`, `docker top`, `docker stats --no-stream web`, modification de la page puis `docker rm -f web`, et volume `-v ~/site:/usr/share/nginx/html:ro`.hello-world, page « Welcome to nginx! », page servie depuis le volume.

Observations :

- `hello-world:latest` : 25,9 kB ; conteneur en état *Exited (0)* après son exécution.
- `nginx:latest` : 243 Mo (contenu 66,8 Mo).
- `docker stats` : 3,445 MiB / 1,604 GiB, 0,21 % CPU, 3 PIDs.
- Distribution du conteneur : Debian 13 (trixie) ; VM : Ubuntu 26.04.1 LTS. Noyau : `7.0.0-38-generic` des deux côtés.
- `ps aux` sur la VM montre le processus `nginx: master` (PID 2370), le même que dans `docker top web`.
- Après `docker rm -f web` puis recréation, le titre est redevenu `Welcome to nginx!` ; avec le volume, la page personnalisée est servie. Un accent mal affiché (« BinÃ´me ») a été corrigé en ajoutant `<meta charset="utf-8">` dans le fichier.

Q1. Les 4 étapes de hello-world. (1) Le client Docker contacte le démon. (2) Le démon télécharge l'image `hello-world` depuis Docker Hub (amd64). (3) Il crée un conteneur à partir de cette image, qui lance l'exécutable produisant le message. (4) Il renvoie cette sortie au client, qui l'affiche dans le terminal.

Q2. Distribution **différente** (Debian 13 dans le conteneur, Ubuntu 26.04 sur la VM), noyau **identique** (`7.0.0-38-generic`). Un conteneur n'embarque pas d'OS invité mais l'application et ses bibliothèques ; la « distribution » vue dedans n'est que l'userland de l'image. Le noyau est unique et partagé avec l'hôte.

Q3. Oui, le processus `nginx: master` (PID 2370) apparaît dans `ps aux` sur la VM. Un conteneur est un simple processus Linux, isolé par les namespaces et limité par les cgroups, et non une machine virtuelle.

Q4. Navigateur Windows → `127.0.0.1:8080` → redirection VirtualBox (hôte:8080 → VM:8080) → redirection Docker (VM:8080 → conteneur:80) → Nginx. Les journaux le confirment : la requête du navigateur arrive depuis `10.0.2.2` (passerelle NAT VirtualBox), celle de `curl` depuis `172.17.0.1` (réseau Docker).

Q5. Un conteneur écrit dans sa propre couche en écriture ; `docker rm` la détruit et le nouveau conteneur repart de l'image intacte : il est éphémère. L'option `-v` monte un dossier de la VM dans le conteneur : les données vivent hors du conteneur et survivent à sa suppression (`:ro` = lecture seule).

Captures des sorties

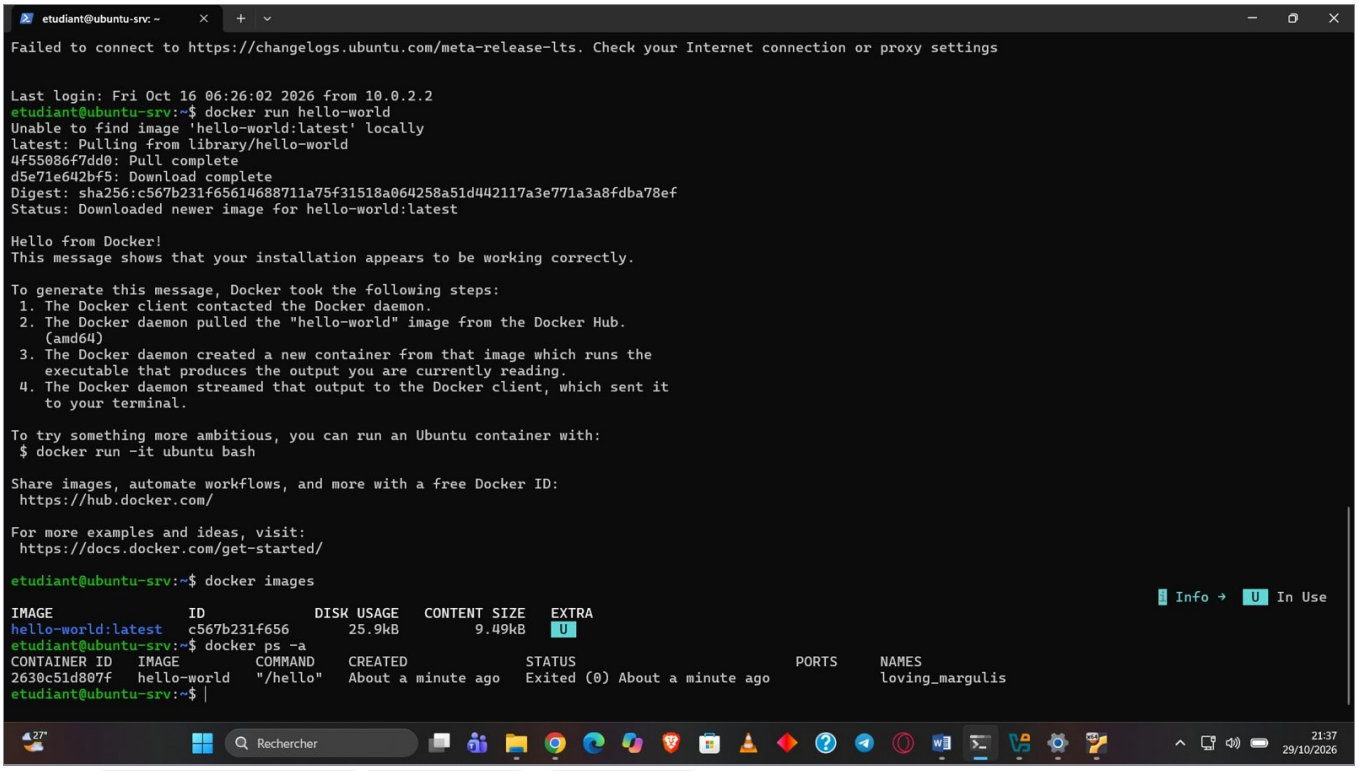


Figure 4. `docker run hello-world`, `docker images` et `docker ps -a`

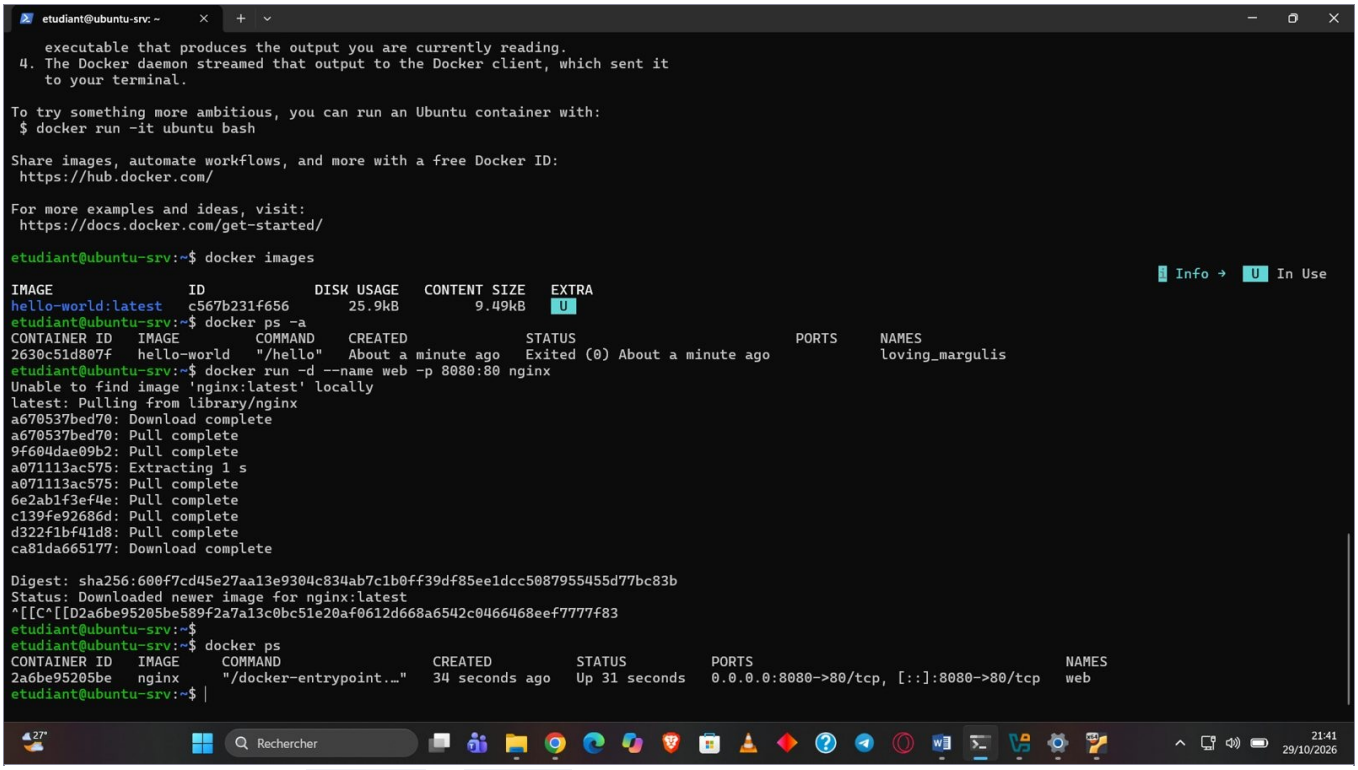


Figure 5. Lancement de Nginx (`-p 8080:80`) et `docker ps`

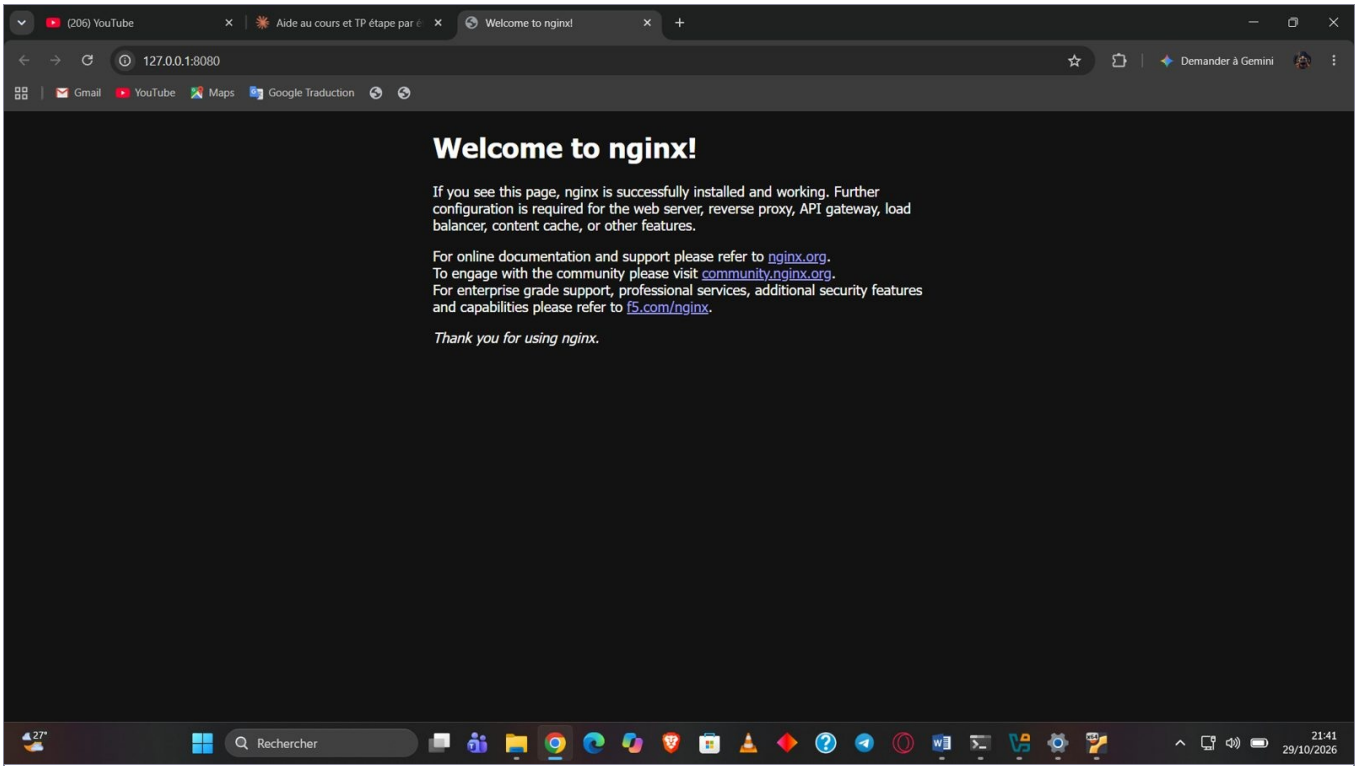


Figure 6. Page « Welcome to nginx! » dans le navigateur Windows (127.0.0.1:8080)

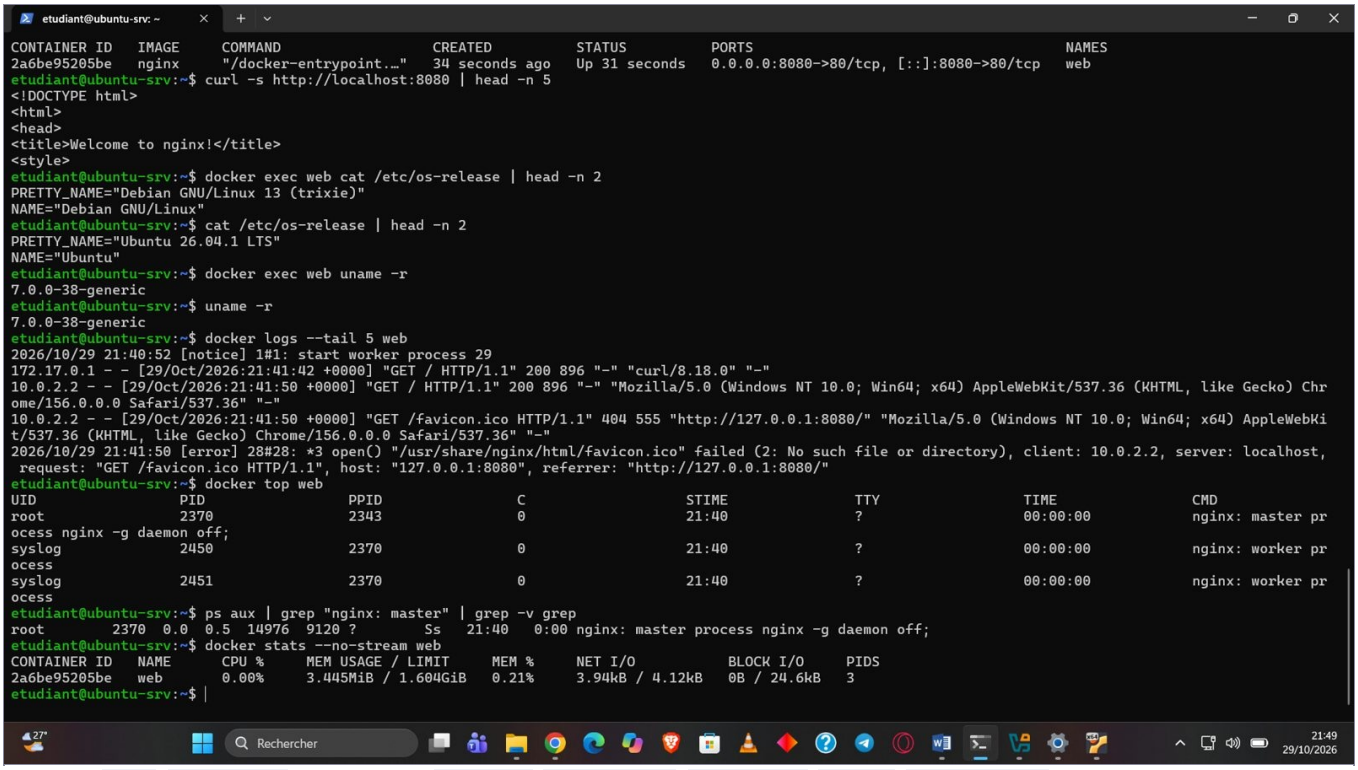


Figure 7. `curl`, `os-release` conteneur/VM, `uname -r`, `docker logs`, `docker top`, `ps aux`, `docker stats`

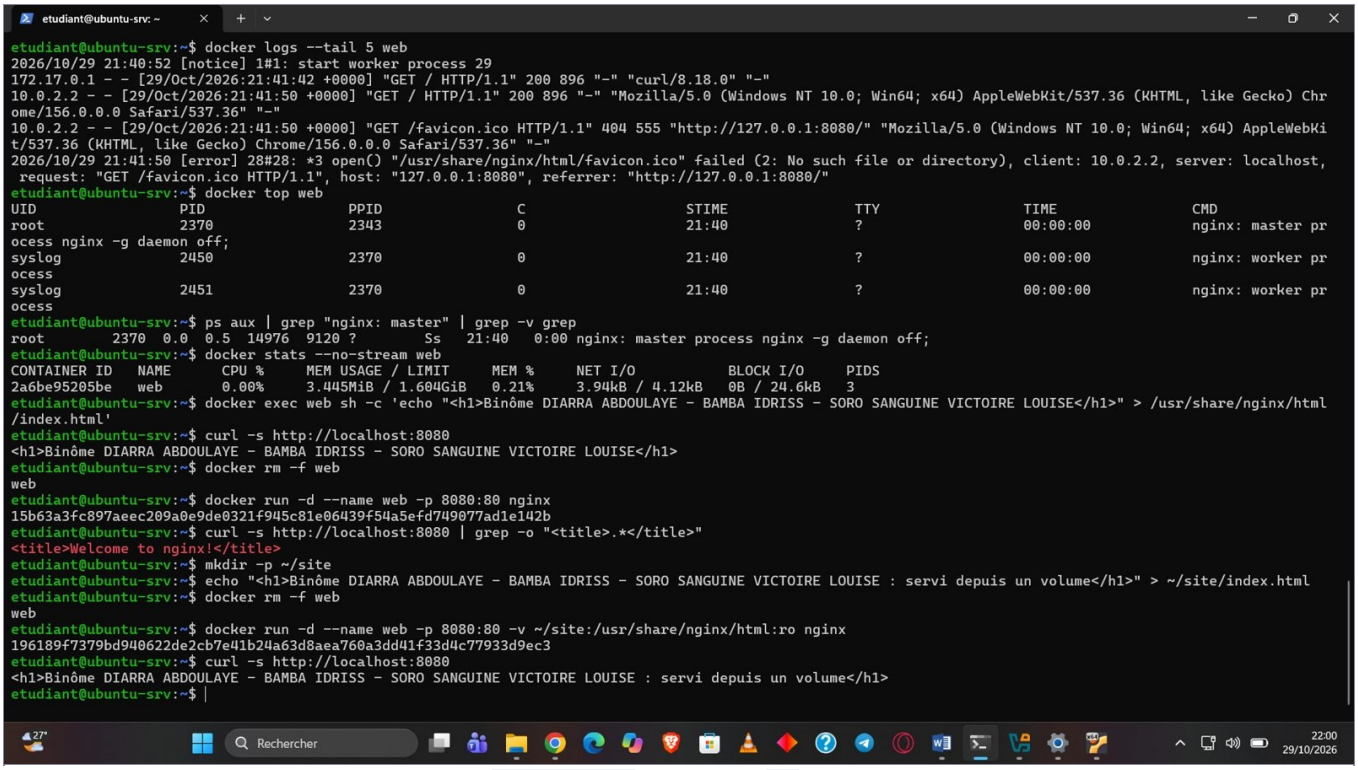


Figure 8. Conteneur éphémère (page perdue après `docker rm -f`) puis volume `-v`

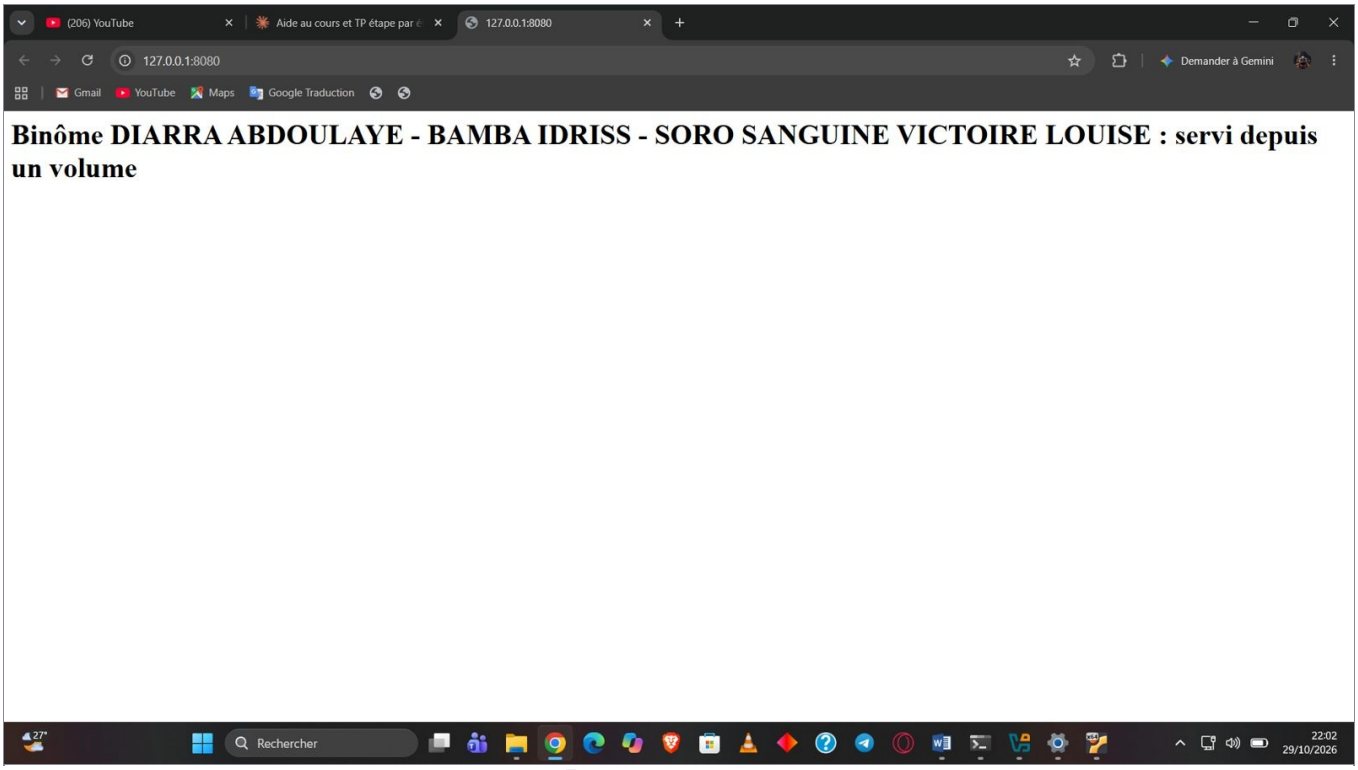


Figure 9. Page servie depuis le volume, accent corrigé avec

Partie 2 — Comparer conteneur et VM

Voir les figures ci-dessous. `docker images`, `time` (×2), `systemd-analyze`, `free -h`, taille du fichier `.vdi`.

Mesure	VM ubuntu-srv	Conteneur nginx	Conteneur alpine
Taille sur disque	6 838 272 Ko ≈ **6,5 Go** (fichier .vdi)	**243 Mo**	**13 Mo**
Temps de démarrage	**1 min 20 s** (1,6 s noyau + 8,3 s initrd + 1 min 10 s userspace)	—	**1,718 s** (2e `time` ; 1er : 2,687 s)
Mémoire utilisée	**492 Mi** (sur 1,6 Gi)	**3,445 MiB**	—
Système	Ubuntu 26.04.1 LTS + noyau	Debian 13 (trixie)	Alpine Linux

Q1. Taille : ≈ ×500 par rapport à alpine (6,5 Go / 13 Mo) et ≈ ×27 par rapport à nginx. Démarrage : ≈ ×47 (80 s / 1,7 s). Mémoire : ≈ ×140 (492 Mi / 3,4 Mi). Les mesures confirment le tableau du cours (Go contre dizaines ou centaines de Mo ; minute contre quelques secondes). Nuance : notre conteneur démarre en 1,7 s et non en 1 s, car la VM dispose de peu de ressources (1,6 Go de RAM). Les deux durées ne mesurent pas non plus la même chose : un système complet pour la VM, un simple processus sur un noyau déjà démarré pour le conteneur.

Q2. Au premier lancement, l'image alpine était déjà téléchargée, mais les fichiers, les programmes de Docker et le noyau n'étaient pas encore en cache mémoire. Au second lancement tout est en cache : 2,687 s contre 1,718 s.

Q3. On choisit une VM pour un autre système d'exploitation ou un autre noyau (par exemple Windows), pour une isolation maximale (code non fiable, plusieurs clients sur la même machine), ou pour migrer tel quel un serveur existant.

Captures des sorties

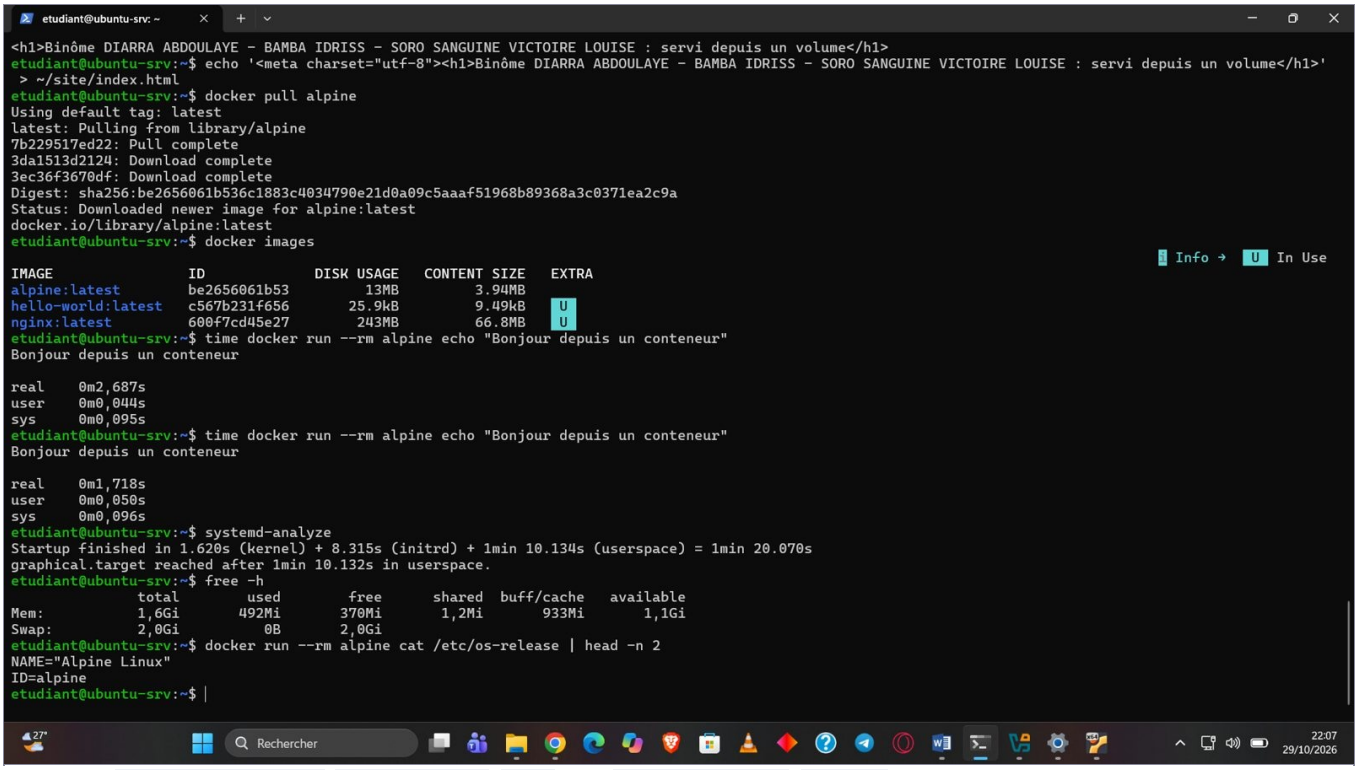


Figure 10. `docker pull alpine`, `docker images`, `time` (×2), `systemd-analyze`, `free -h`

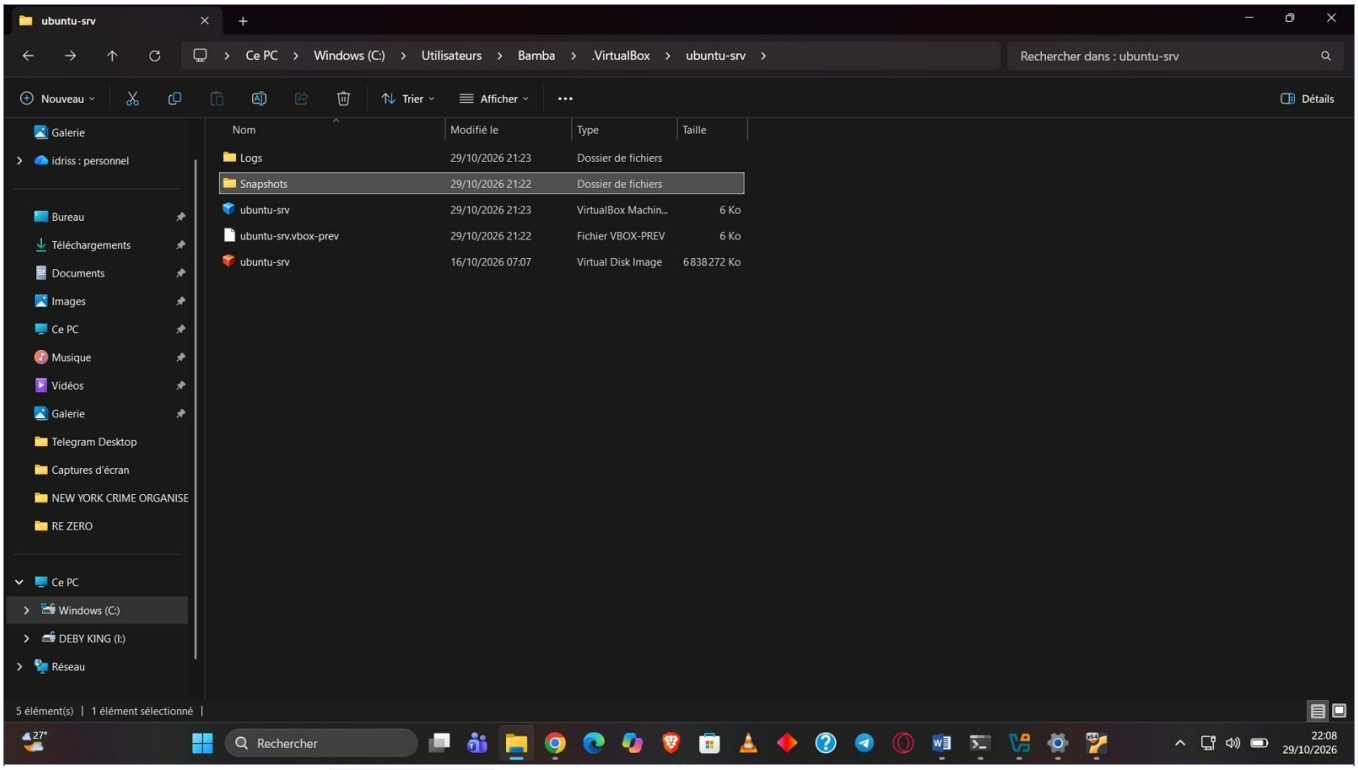


Figure 11. Taille du fichier disque `ubuntu-srv.vdi` de la VM (6 838 272 Ko)

Partie 3 — Conteneuriser l'API fil rouge

Fichiers dans `api/` : `app.py` (routes `/`, `/sante`, puis `/version`), `requirements.txt` (`flask>=3,<4`), `Dockerfile`, `.dockerignore`.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Voir les figures ci-dessous. `docker build`, `docker history`, les `curl`, les trois copies, le second build.

- `docker build -t api:1.0 .` : 48,0 s (dont `pip install` : 28,6 s). `docker images api` : **198 Mo** (contenu 48,3 Mo).
- `docker run -d --name api -p 5000:5000 -e BINOME="DIARRA-BAMBA-SORO" api:1.0` : `{"binome":"DIARRA-BAMBA-SORO","conteneur":"d8394b354faf","message":"API fil rouge"}` ; `/sante` renvoie `{"statut":"ok"}`.
- Trois copies (ports 5001, 5002, 5003) : champ `conteneur` = `fea91495ba61`, `f3e5c8a15902`, `f6193da1192d`.
- Après ajout de `/version`, `docker build -t api:1.1 .` : 39,3 s (dont 23,3 s de lecture des métadonnées de l'image de base, lente avec la connexion). Étapes `WORKDIR`, `COPY requirements.txt` et `RUN pip install` : **CACHED** (0,0 s) ; seule `COPY . .` est refaite (1,3 s). `curl …/version` renvoie `{"version":"1.1"}`.

Q1. `api:1.0` pèse 198 Mo. Six couches viennent de notre Dockerfile : `WORKDIR` (8,19 kB), `COPY requirements.txt` (12,3 kB), `RUN pip install` (15,3 Mo, qui contient Flask), `COPY . .` (20,5 kB), plus `EXPOSE` et `CMD` (0 B, métadonnées). Les autres viennent de `python:3.12-slim` (dont 87,7 Mo pour la base Debian) : l'image de base pèse bien plus que notre application.

Q2. Docker met chaque instruction en cache tant que ses entrées ne changent pas ; quand une couche change, toutes les suivantes sont refaites. En copiant d'abord `requirements.txt` et en installant les dépendances avant de copier le code, la couche lente est réutilisée quand seul `app.py` change : lors du build de `api:1.1`, `pip install` apparaît `CACHED` (0,0 s contre 28,6 s au premier build) et seule `COPY . .` est refaite (1,3 s).

Q3. Trois identifiants différents : chaque conteneur a son propre nom d'hôte. L'expérience illustre l'**élasticité** (lancer vite autant d'instances que nécessaire à partir d'une seule image, puis les supprimer) et la mutualisation des ressources (même noyau, même machine).

Q4. L'image doit rester identique partout ; ce qui change selon l'environnement (nom, adresses, mots de passe, clés) s'injecte au lancement avec `-e`, sans reconstruire l'image. Un secret écrit dans le code ou dans l'image serait lisible par quiconque récupère l'image : on l'injecte à l'exécution, puis avec un gestionnaire de secrets.

Captures des sorties

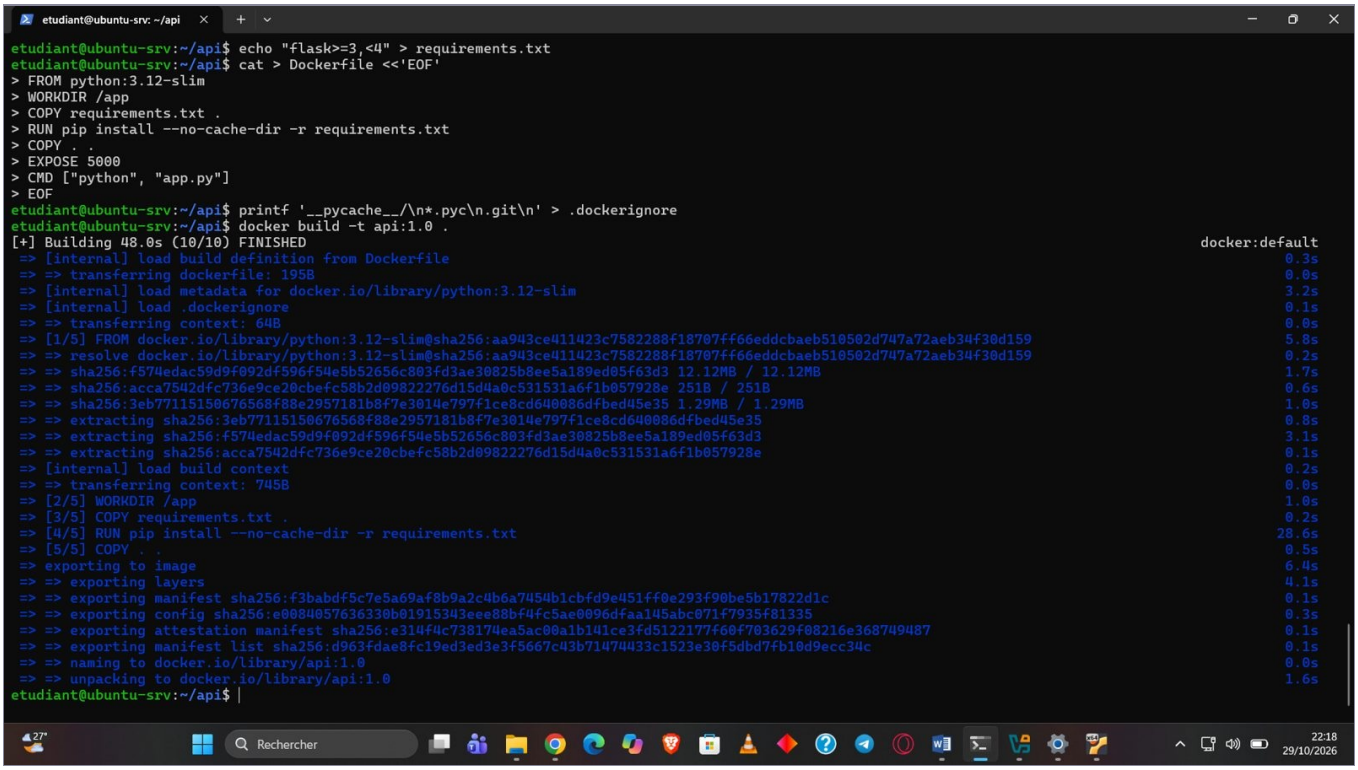


Figure 12. `docker build -t api:1.0 .` : création du Dockerfile et build (48,0 s)

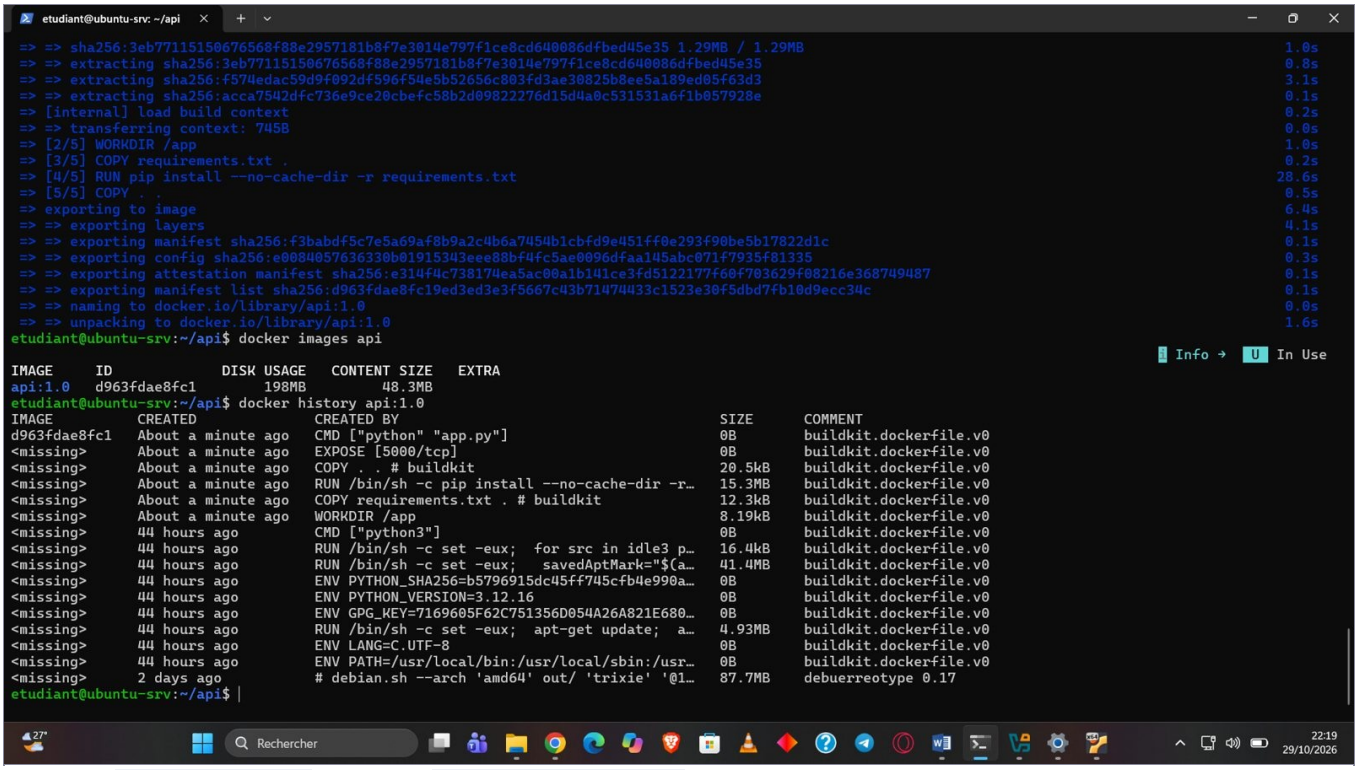


Figure 13. `docker images api` (198 Mo) et `docker history api:1.0`

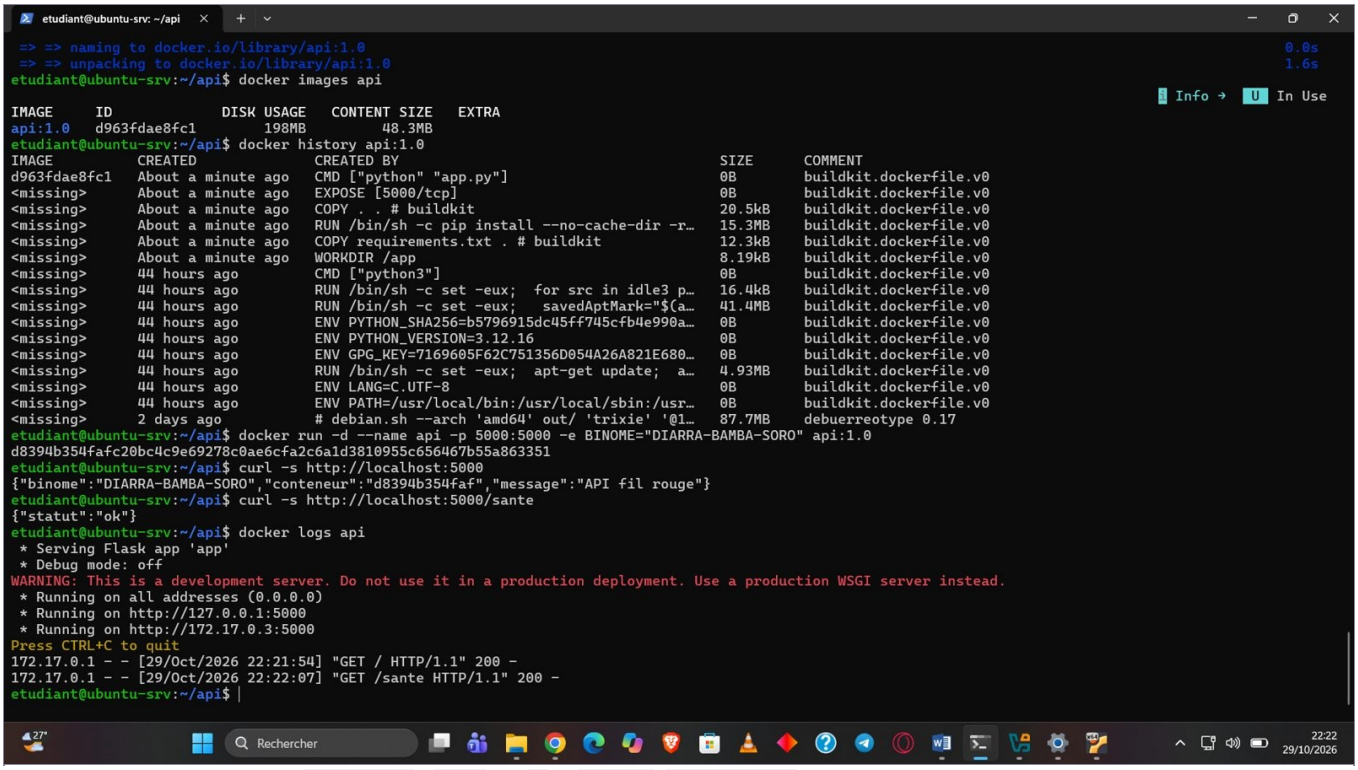


Figure 14. Lancement de l'API (`-e BINOME`), `curl` sur `/` et `/sante`, `docker logs`

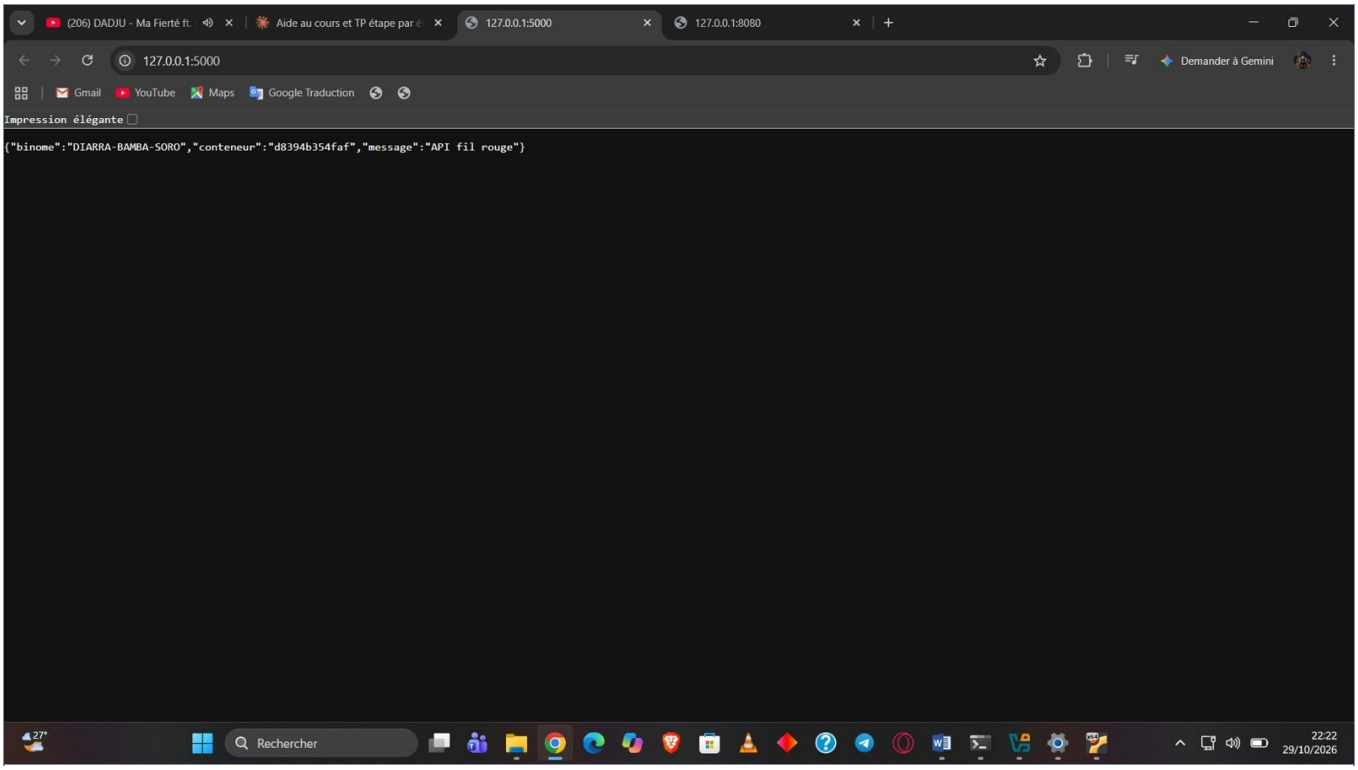


Figure 15. L'API dans le navigateur Windows (127.0.0.1:5000)

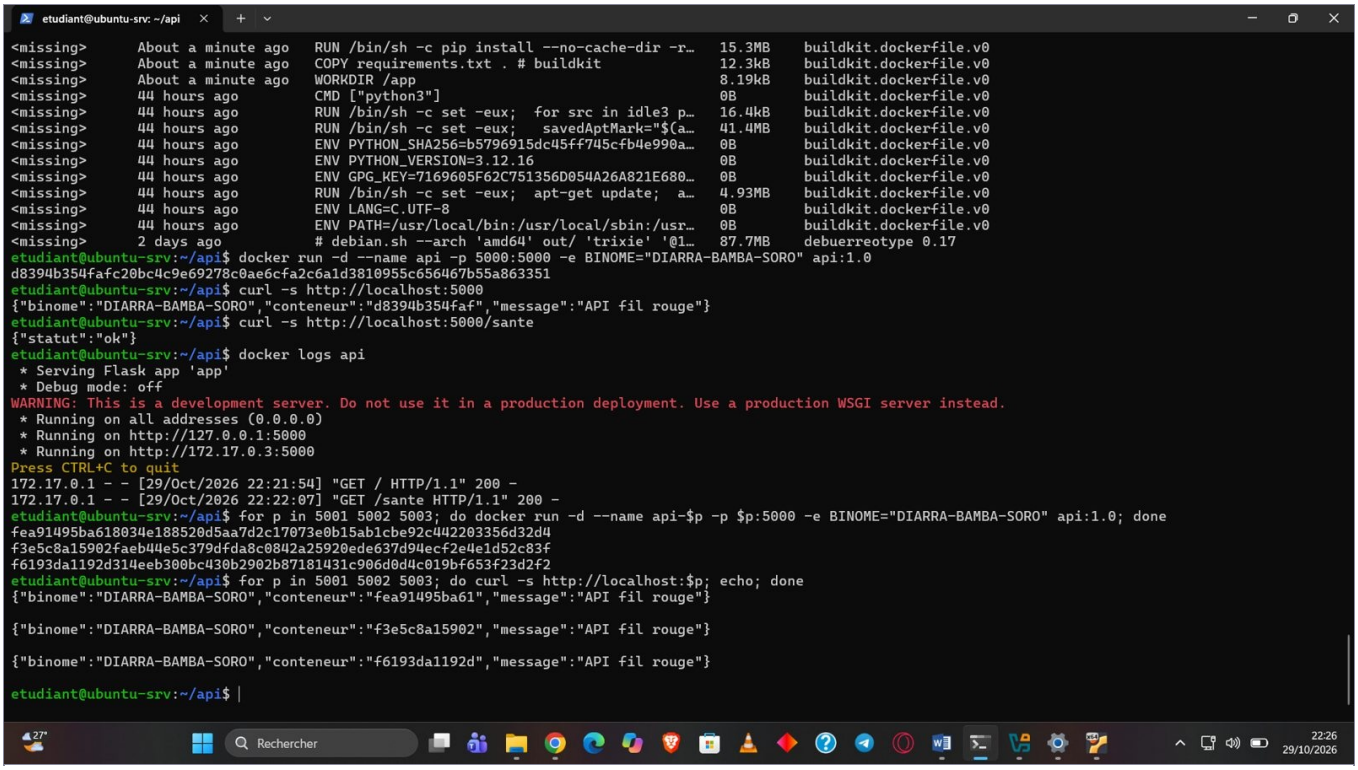


Figure 16. Trois conteneurs depuis la même image : trois identifiants différents

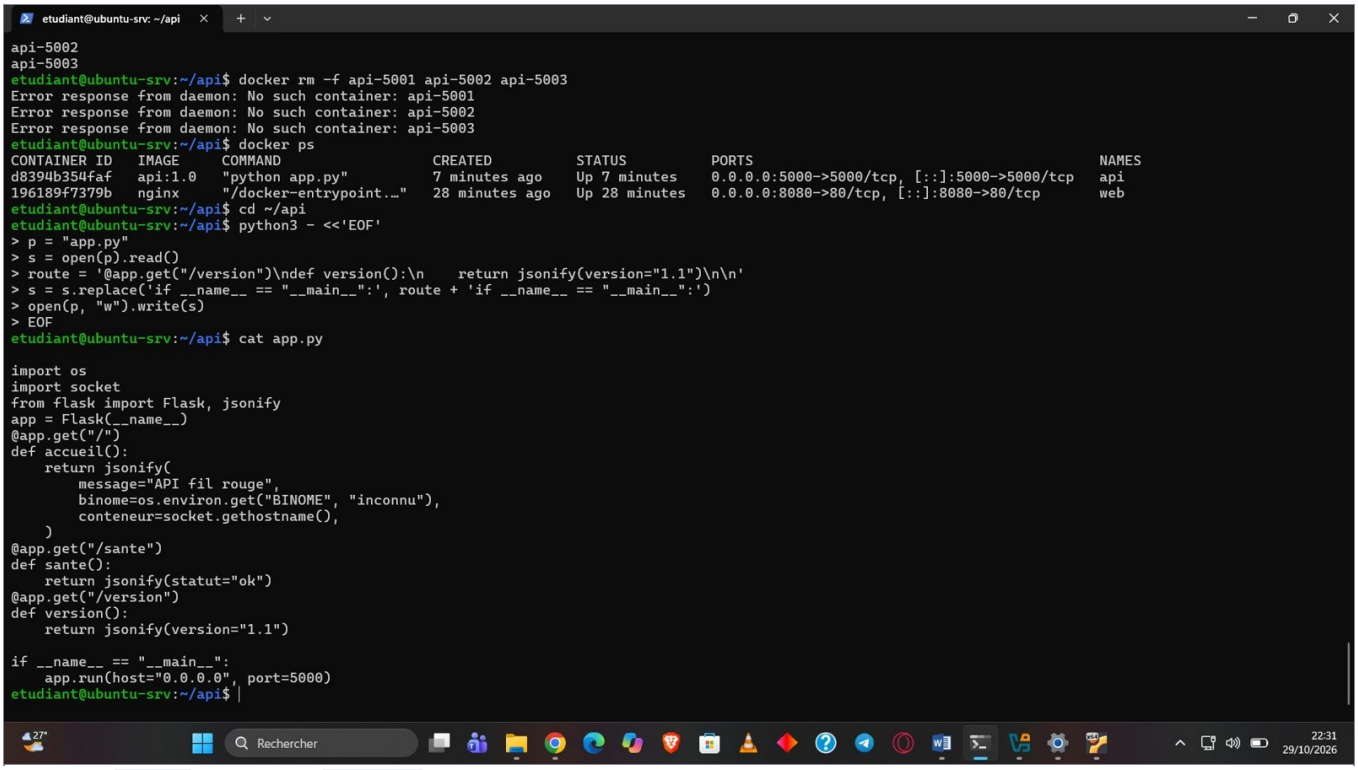


Figure 17. `app.py` avec la route `/version`

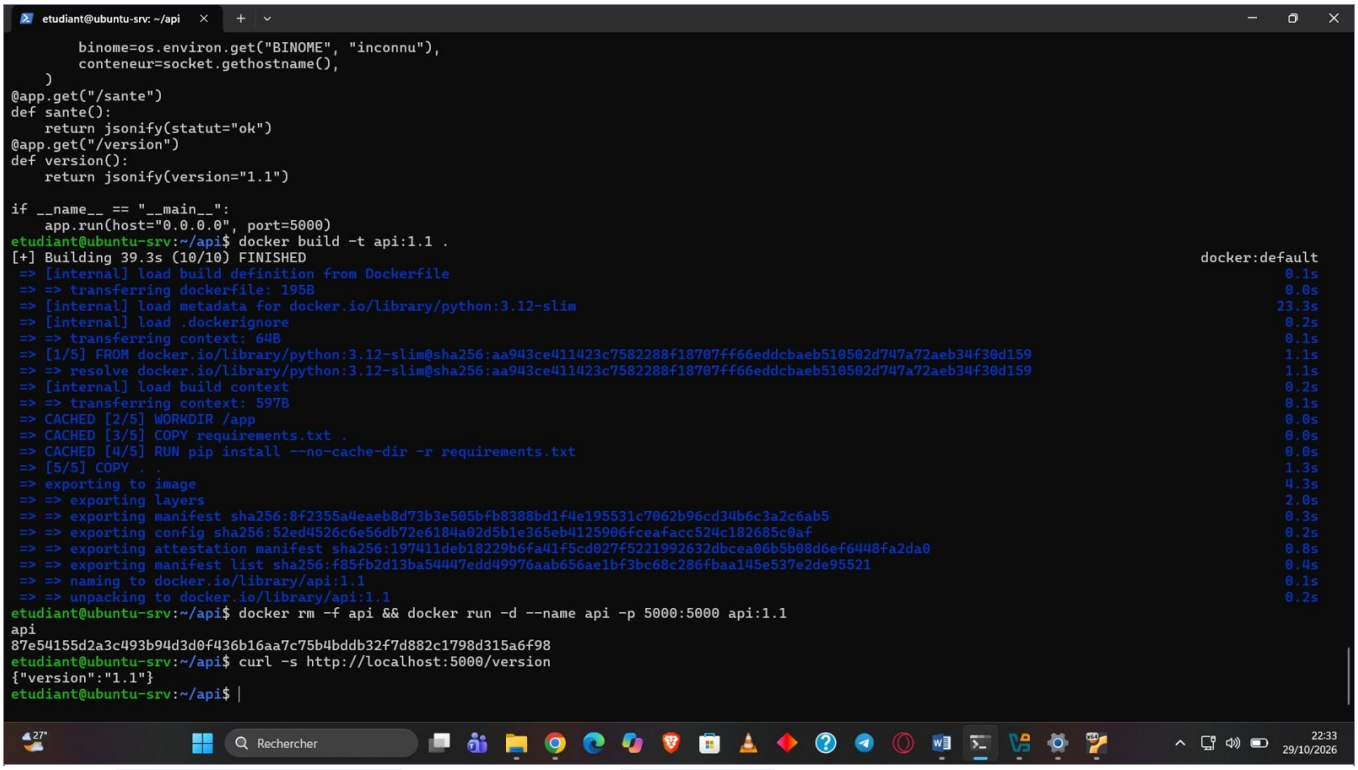


Figure 18. `docker build -t api:1.1 .` : étapes CACHED, puis `curl /version`

Partie 4 — Publier l'image et nettoyer

Registre choisi : **Docker Hub**, compte `bambaidrissshafihi`. Connexion avec un jeton d'accès (Read & Write), envoyé par `--password-stdin`. Commandes : `docker tag api:1.1 bambaidrissshafihi/api-fil-rouge:1.1`, `docker push bambaidrissshafihi/api-fil-rouge:1.1` (digest `sha256:f85fb2d1…e95521`, size 856). `docker push`, page du dépôt sur Docker Hub.

Vérification : après `docker rm -f api` et `docker rmi` des copies locales, puis `docker logout`, `docker run -d --name api -p 5000:5000 bambaidrissshafihi/api-fil-rouge:1.1` a téléchargé l'image depuis le registre (même digest) ; `curl …/version` renvoie `{"version":"1.1"}`. Le téléchargement a fonctionné sans connexion : le dépôt est **public**.

Nettoyage : `docker rm -f $(docker ps -aq)`, `docker system df`, `docker image prune -a -f`, `docker system df`.

	Images	Taille	Reclaimable
Avant `prune`	5	352,3 Mo	86,98 Mo (24 %)
Après `prune`	0	149,7 Mo	149,7 Mo (100 %)

`prune` a indiqué **40,94 Mo** récupérés ; le cache de build (110,3 Mo) reste à nettoyer avec `docker builder prune`.

Q1. Adresse : `docker.io/bambaidrissshafihi/api-fil-rouge:1.1`. Dépôt **public** (téléchargé sans authentification).

Q2. Un tag de version désigne une image précise et reproductible : on sait ce qui tourne, et on peut revenir à `1.0` en cas de problème. `latest` est un tag mobile qui peut pointer vers autre chose du jour au lendemain, ce qui rend les déploiements imprévisibles.

Q3. `docker image prune -a -f` a libéré **40,94 Mo** (taille des images passée de 352,3 Mo à 149,7 Mo, les couches restantes étant désormais entièrement récupérables). Dans le cloud, les registres facturent le stockage : les anciennes images s'accumulent, coûtent de l'argent et gardent des vulnérabilités anciennes, d'où la nécessité de nettoyer aussi les registres.

Sécurité : le jeton d'accès n'a pas été mis dans le dépôt ni dans l'image ; il a été supprimé de la VM (`docker logout`) et doit être révoqué sur Docker Hub.

Captures des sorties

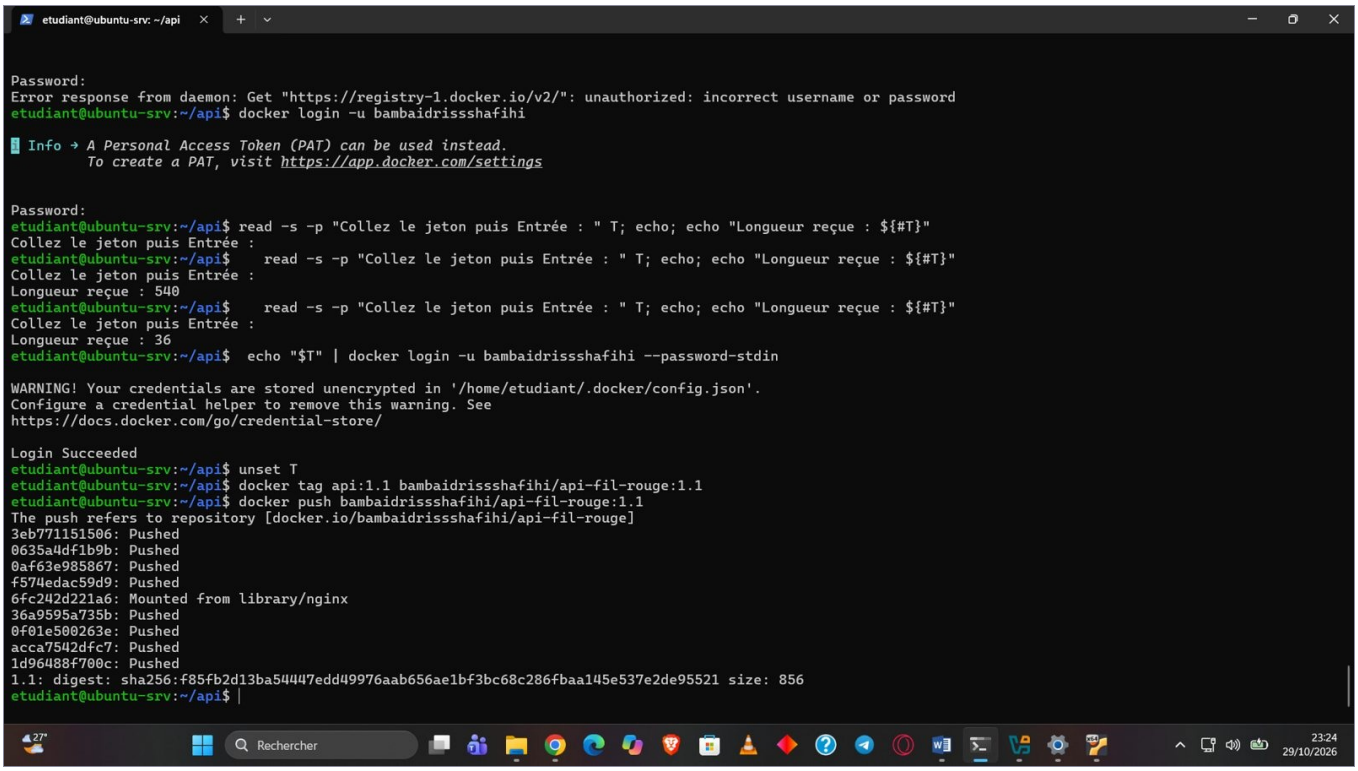


Figure 19. `docker login` (Login Succeeded) puis `docker tag` et `docker push` (digest sha256:f85fb2d1…)

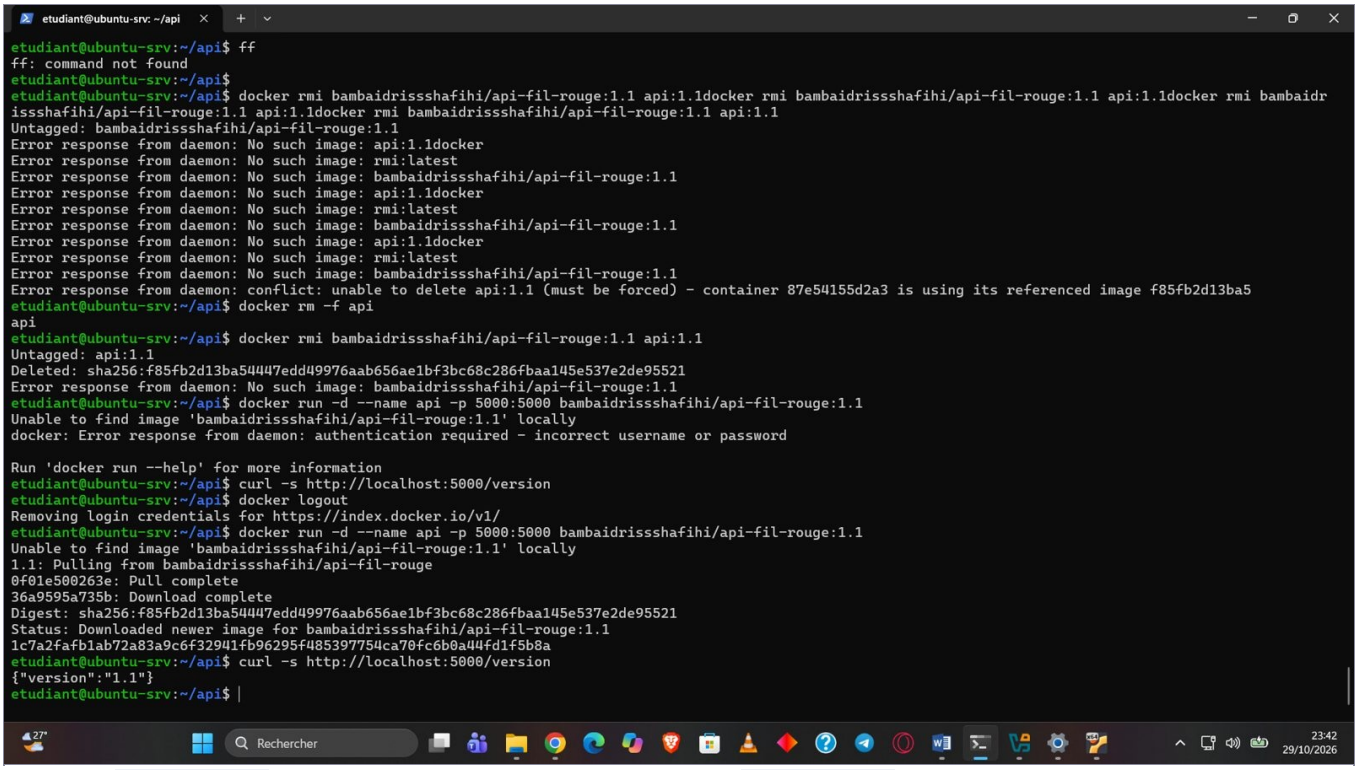


Figure 20. `docker logout`, téléchargement de l'image depuis Docker Hub et `curl /version`

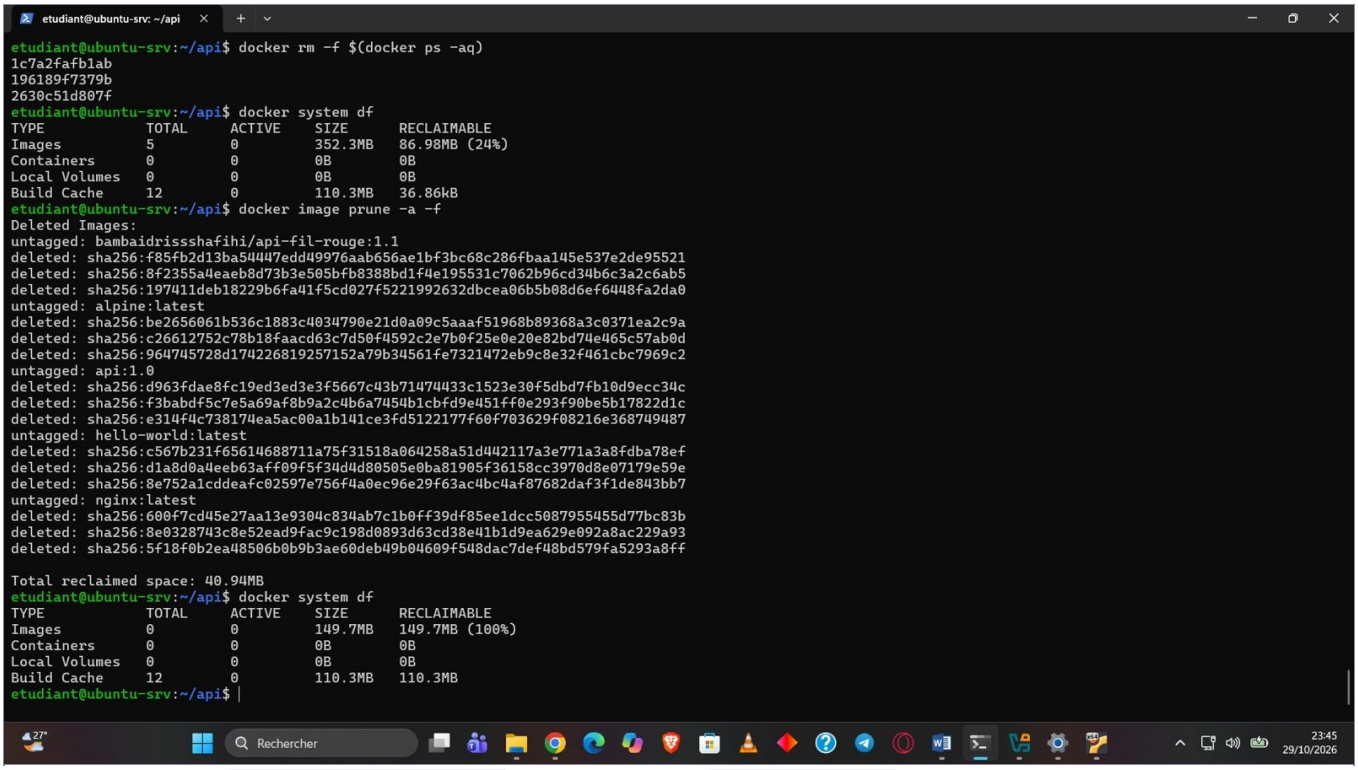


Figure 21. Nettoyage : `docker image prune -a -f` (40,94 Mo libérés) et `docker system df`

Difficultés rencontrées

Problème	Cause	Solution
`usermod` affichait son aide	Commande incomplète (`docker` ou l'utilisateur manquait), aggravée par des gels de la VM (« soft lockup »)	`sudo usermod -aG docker etudiant`, vérifié avec `getent group docker`
SSH : *host identification has changed*	Ancienne clé mémorisée pour `[127.0.0.1]:2222`	`ssh-keygen -R "[127.0.0.1]:2222"`
SSH : *Connection reset*	Service `ssh` inactif et désactivé	`sudo systemctl enable --now ssh`
`docker login` : *registry-1.docker.io ... connection refused*	Résolution DNS défaillante dans la VM	Redémarrage de `systemd-resolved`
`docker login` : *incorrect username or password*	Jeton non reçu (collage perdu dans le terminal)	Contrôle de la longueur avec `read -s`, puis `--password-stdin`
`docker run` : *authentication required*	Identifiant périmé conservé dans `config.json`	`docker logout`, puis pull anonyme

Captures des sorties

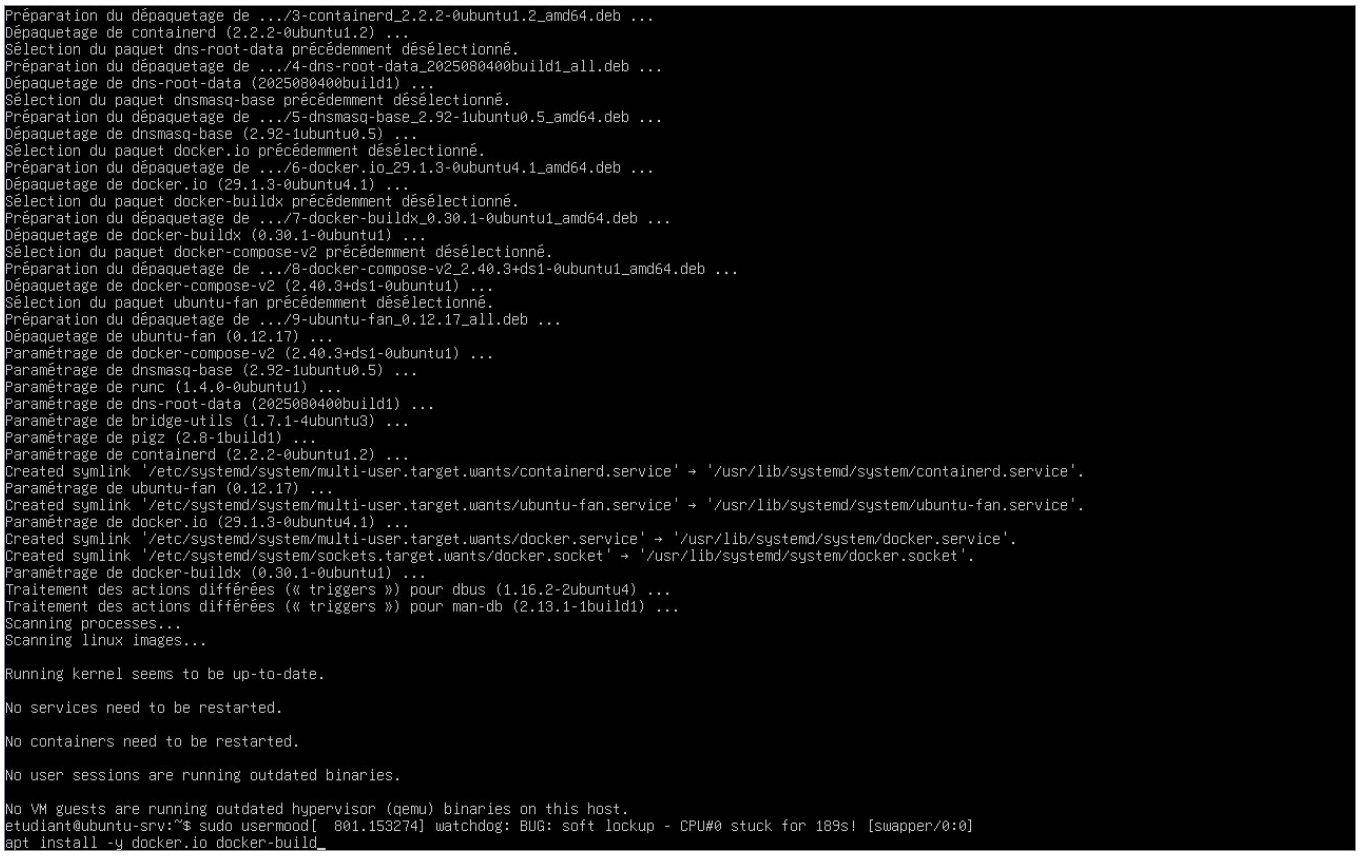


Figure 22. Ligne de commande corrompue par un « soft lockup » de la VM

```
Ubuntu 26.04.1 LTS ubuntu-srv tty1

ubuntu-srv login: etudiant
Password:
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-38-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of jeu. 29 oct 2026 16:12:27 UTC

  System load:             0.81
  Usage of /:              52.5% of 11.21GB
  Memory usage:            19%
  Swap usage:              0%
  Processes:               125
  Users logged in:         0
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe55:89e8


La maintenance de sécurité étendue pour Applications n'est pas activée.

41 mises à jour peuvent être appliquées immédiatement.
10 de ces mises à jour sont des mises à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

Activez ESM Apps pour recevoir des futures mises à jour de sécurité supplémentaires.
Visitez https://ubuntu.com/esm ou executez : sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings


etudiant@ubuntu-srv:~$ docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.24.13
 Git commit:        29.1.3-0ubuntu4.1
 Built:             Wed Apr 29 16:40:20 2026
 OS/Arch:           linux/amd64
 Context:           default
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
etudiant@ubuntu-srv:~$ docker info | grep -E "Server Version|Storage Driver|Cgroup|Kernel Version"
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
etudiant@ubuntu-srv:~$ _
```

Figure 23. `permission denied … docker.sock` : utilisateur pas encore dans le groupe docker

```
[ 2474.294515] rcu: rcu_preempt kthread starved for 213066 jiffies! g90229 f0x2 RCU_GP_WAIT_FQS(5) ->state=0x200 ->cpu=0
[ 2474.294835] rcu: ‡Unless rcu_preempt kthread gets sufficient CPU time, OOM is now expected behavior.
[ 2474.295146] rcu: RCU grace-period kthread stack dump:
[ 2474.297685] rcu: Stack dump where RCU GP kthread last ran:
 tdre
Command 'tdre' not found, did you mean:
  command 'tdr' from deb devtodo (0.1.20+git20200830.0ad52b0-3build1)
  command 'tre' from deb tre-command (0.4.0-7)
  command 'tde' from deb devtodo (0.1.20+git20200830.0ad52b0-3build1)
Try: sudo apt install <deb name>
etudiant@ubuntu-srv:~$ sudo usermod -aG etudiant
Utilisation : usermod[options] LOGIN

Options :
  -a, --append                  ajouter l'utilisateur au GROUPS supplémentaire
                                      mentionné par l'option -G sans supprimer
                                      l'utilisateur des autres groupes
  -b, --badname                 allow bad names (DEPRECATED)
  -c, --comment COMMENT         nouvelle valeur du champ GECOS
  -d, --home HOME_DIR           nouveau répertoire personnel du compte utilisateur
 -e, --expiredate EXPIRE_DATE   définir la date d'expiration du compte avec EXPIRE_DATE
  -f, --inactive INACTIVE       définir le mot de passe inactif après expiration
                                à INACTIVE
  -g, --gid GROUP               forcer l'utiisation de GROUP comme groupe primaire
  -G, --groups GROUPS           nouvelle liste de groupes GROUPS supplémentaires
  -h, --help                    afficher ce message d'aide et quitter
  -l, --login NEW_LOGIN         nouvelle valeur pour le nom de connexion
  -L, --lock                    verrouiller le compte utilisateur
  -m, --move-home               déplacer le contenu du répertoire personnel dans le
                                    nouvel emplacement (à utiliser seulement avec -d)
  -o, --non-unique              autoriser l'utilisation d'UID dupliqué (non-unique)
  -p, --password PASSWORD       utiliser un mot de passe chiffré pour le nouveau mot de passe
  -P, --prefix PREFIX_DIR       préfixe du répertoire où sont situés les fichiers etc/*
  -r, --remove                   supprimer l'utilisateur uniquement du GROUPS supplémentaire
                                      mentionné par l'option -G sans supprimer
                                      l'utilisateur des autres groupes
 -R, --root CHROOT_DIR          répertoire dans lequel faire un chroot
  -s, --shell SHELL             nouveau shell de login pour le compte utilisateur
  -u, --uid UID                 nouvel UID pour le compte utilisateur
  -U, --unlock                  déverrouiller le compte utilisateur
  -v, --add-subuids FIRST-LAST  ajouter une gamme d'UID subordonnés
  -V, --del-subuids FIRST-LAST  supprimer la gamme des UID subordonnés
  -w, --add-subgids FIRST-LAST  ajouter une gamme de GID subordonnés
  -W, --del-subgids FIRST-LAST supprimer la gamme des GID subordonnés
 -Z, --selinux-user SEUSER   nouveau mappage d'utilisateur SELinux pour le compte utilisateur
     --selinux-range SERANGE   nouvelle plage MLS de SELinux pour le compte utilisateur

etudiant@ubuntu-srv:~$ getent group docker
docker:x:107:
etudiant@ubuntu-srv:~$
```

Figure 24. `usermod` sans le nom du groupe : affichage de l'aide, puis commande correcte

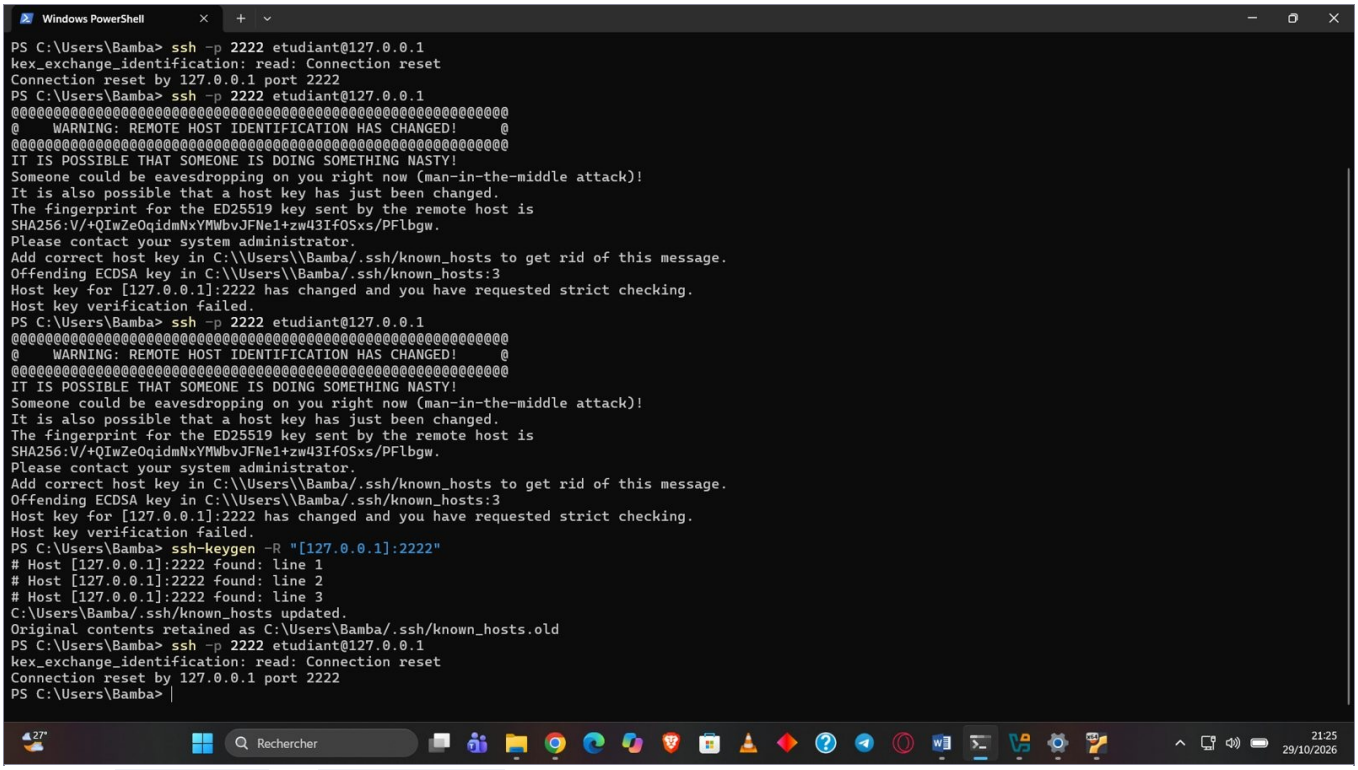


Figure 25. `ssh-keygen -R` puis `Connection reset` (service SSH arrêté)

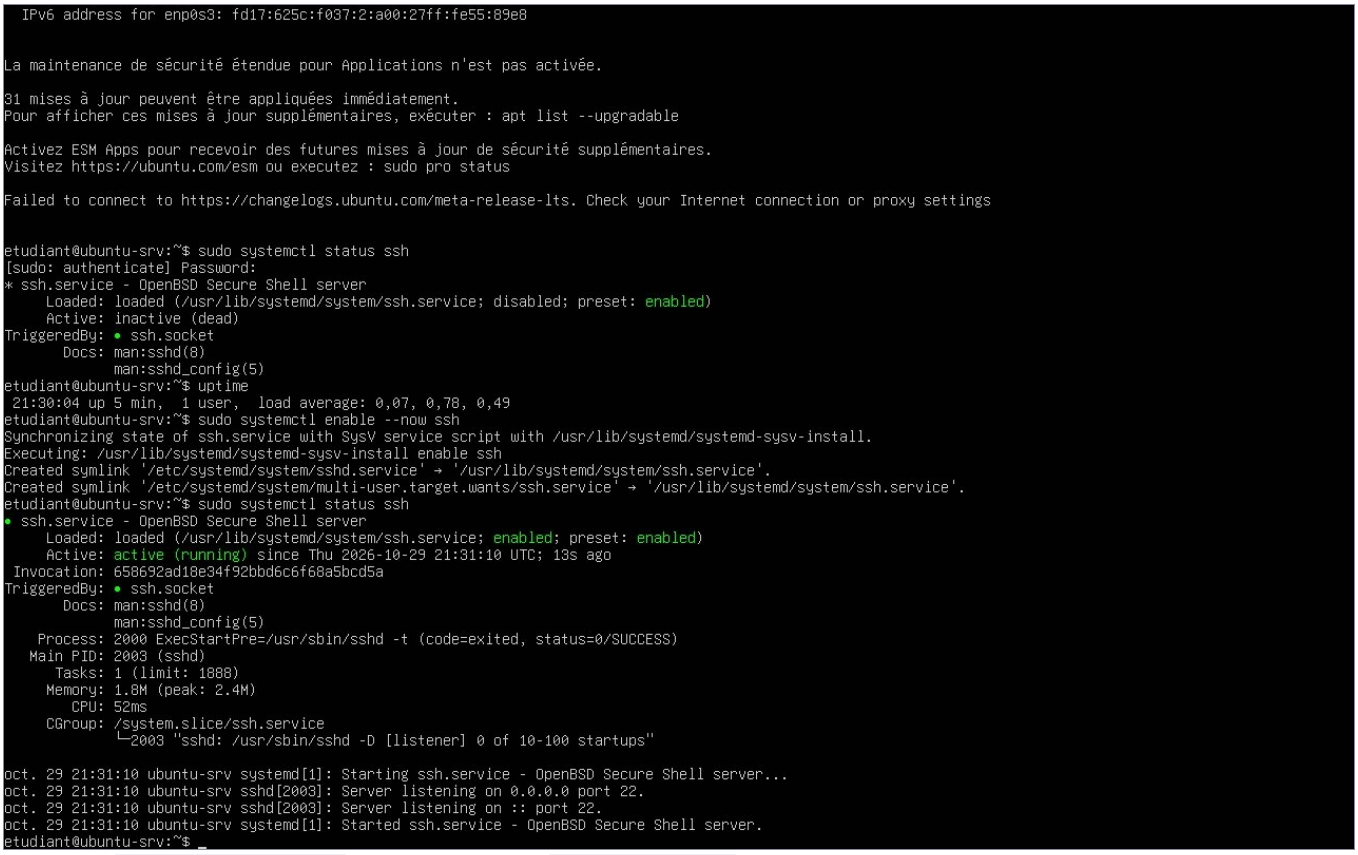


Figure 26. `systemctl status ssh` : service inactif, puis `enable --now ssh`

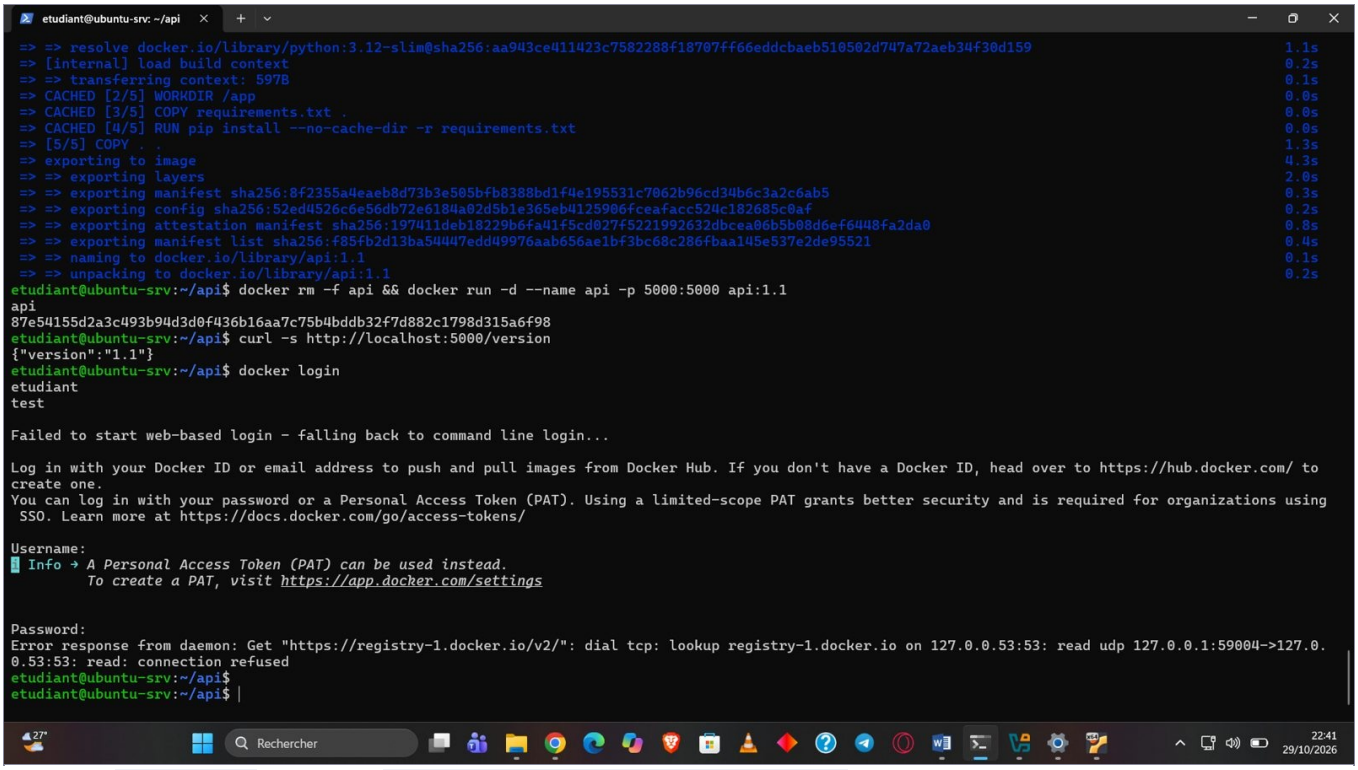


Figure 27. `docker login` : erreur DNS `registry-1.docker.io … connection refused`

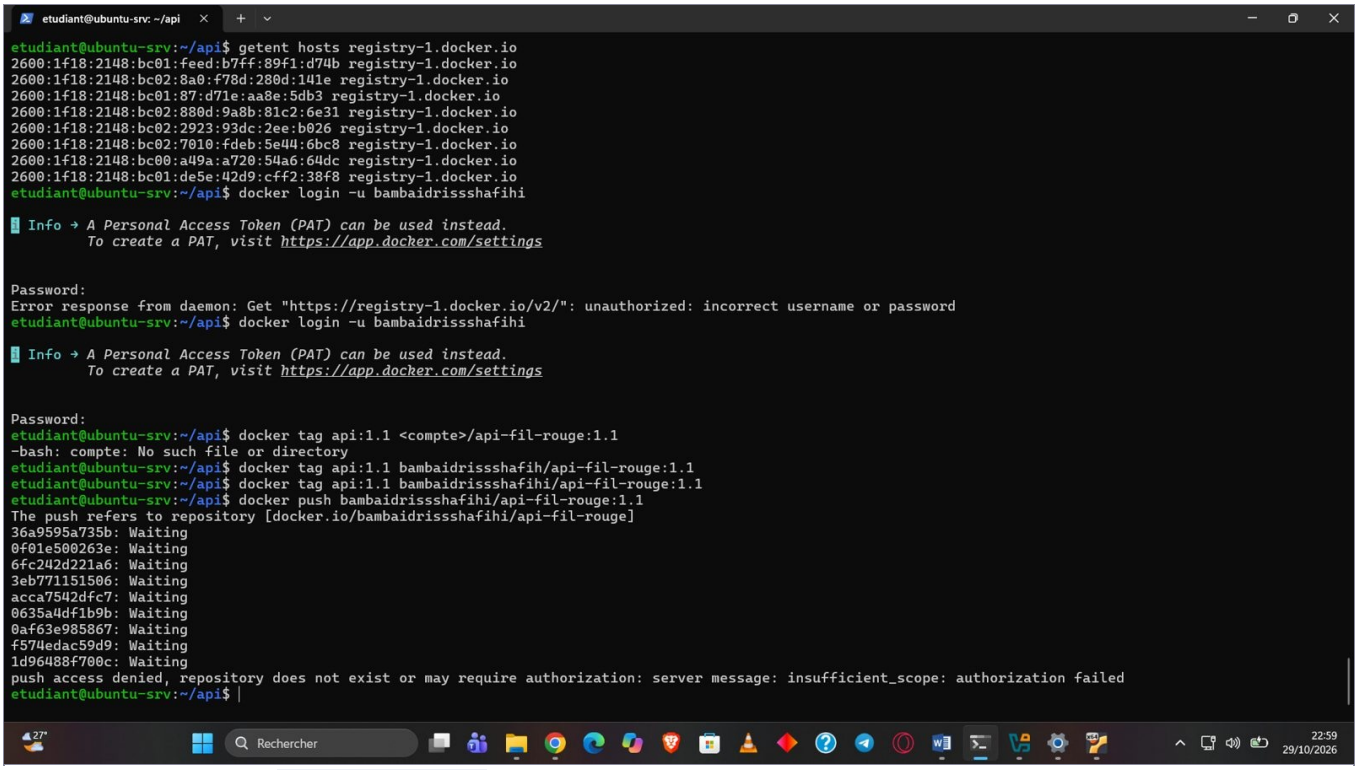


Figure 28. DNS réparé, puis `push access denied` faute de connexion valide

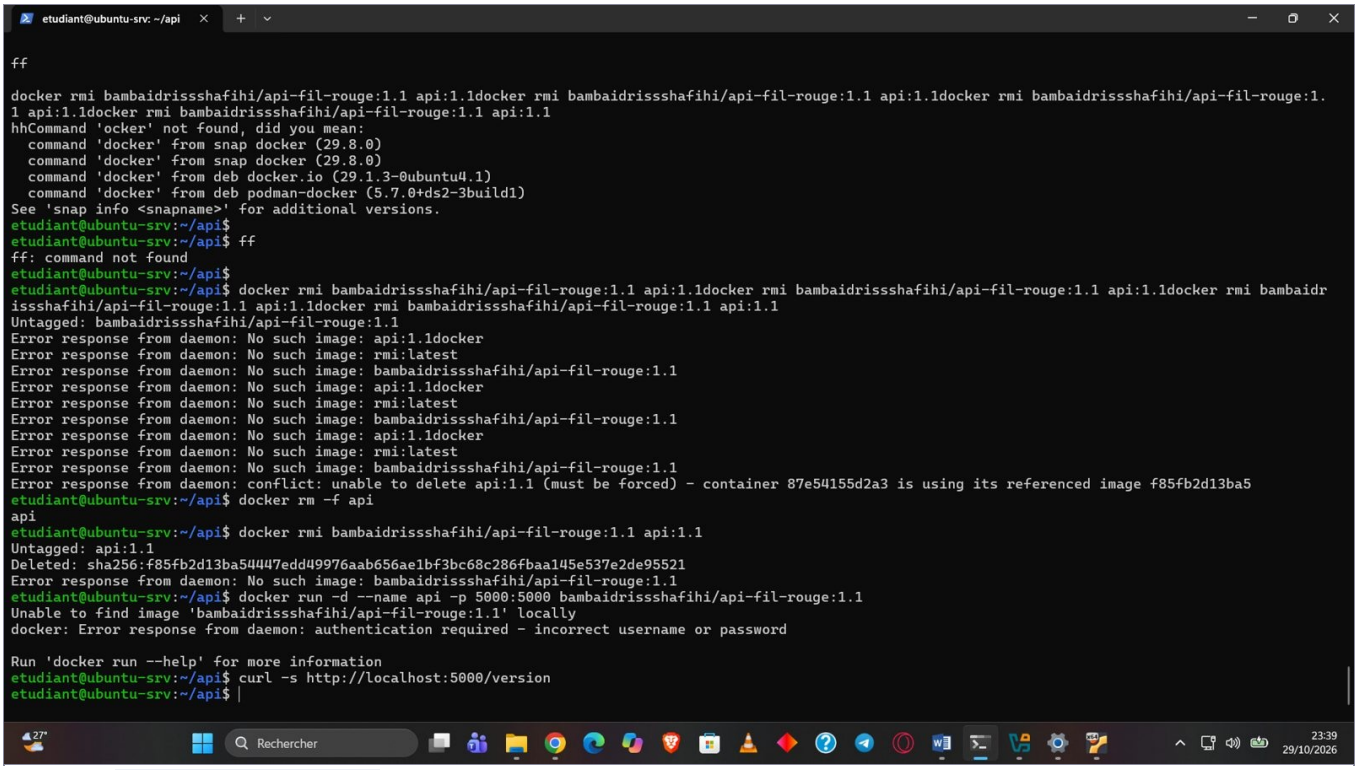


Figure 29. `authentication required` : identifiant périmé conservé dans la configuration Docker

Conclusion

Un conteneur est un processus isolé qui partage le noyau de l'hôte : nos mesures montrent qu'il est bien plus léger (13 Mo contre 6,5 Go) et plus rapide à démarrer (1,7 s contre 1 min 20 s) qu'une VM, au prix d'une isolation plus faible. Nous avons parcouru le cycle complet Dockerfile → image → registre → conteneur ; l'image publiée sera réutilisée en séance 4 (Docker Compose) et en séance 6 (Docker sur une VM EC2).